

Conclusions i propostes a futur

Bot Detector — Disseny i Anàlisi d'Algoritmes Avançats



Asignatura: Disseny i Anàlisi d'Algoritmes Avançats

Centro: ENTI · Universitat de Barcelona

Alumno: Víctor Castro Quesada i Oscar Fernandez Andujar

Profesor: Marti de Ferrer

1. Anàlisi teòrica de complexitat

Variables utilitzades:

- **N**: nombre de línies del log.
- **V**: nombre d'events guardats a la finestra en un moment donat.
- **P**: nombre de ports que ha provat una IP.
- **n**: nombre de nodes de l'arbre de subxarxes.

1.1. Finestra lliscant (Cua)

Funció	Millor	Mitjà	Pitjor	Explicació
anadir()	$O(1)$	$O(1)$	$O(V)$	Afegir és $O(1)$; netejar pot esborrar varis antics
contar_ip()	$O(V)$	$O(V)$	$O(V)$	Recorre tota la cua sumant coincidències
tamano()	$O(1)$	$O(1)$	$O(1)$	len() d'un deque és directe

L'operació anadir() és $O(1)$ en la majoria de casos perquè afegir al final d'un deque és instantani. El pitjor cas $O(V)$ passa quan cal esborrar molts events antics de cop, encara que cada event només s'esborra una vegada en tota la seva vida (amortitzat).

1.2. Arbre de subxarxes (Arbre N-ari)

Funció	Millor	Mitjà	Pitjor	Explicació
insertar()	$O(1)$	$O(1)$	$O(1)$	Sempre baixa 4 nivells (4 octets)
buscar_subredes()	$O(1)$	$O(n)$	$O(n)$	Recorre tots els nodes de l'arbre

Insertar una IP sempre costa el mateix: $O(1)$. Això és perquè una IP té sempre 4 parts (octets), així que l'arbre mai no té més de 4 nivells de profunditat. Buscar les subxarxes atacades recorre l'arbre sencer, per això és $O(n)$.

1.3. Analitzadors (POO)

Funció	Millor	Mitjà	Pitjor	Explicació
AnalizadorLogin.analizar()	$O(L)$	$O(L)$	$O(L)$	Busca text i parteix la línia per espais
AnalizadorPuertos._distintos()	$O(1)$	$O(P)$	$O(P)$	Recursiu: recorre la llista de ports

L és la longitud de la línia. La funció recursiva _distintos() recorre la llista de ports un a un, per això és $O(P)$ en el cas normal i pitjor.

1.4. Sistema complet

```
Cost total = O(N · A)
```

```
N = nombre de línies
```

```
A = nombre d'analitzadors (en el nostre cas, 2)
```

Com que A és una constant petita (2 analitzadors), el cost real és lineal: $O(N)$. Si doblem la mida del log, el temps de procés també es dobla.

1.5. Recursivitat en el projecte

- **Arbre de subxarxes:** les funcions `_insertar()` i `_buscar()` baixen per l'arbre cridant-se a si mateixes, un nivell per cada octet de la IP.
- **Anàlisi de ports:** la funció `_distintos()` recorre la llista de ports de forma recursiva, amb un cas base clar (quan arriba al final de la llista).

2. Anàlisi de temps mitjans

El sistema ha estat provat amb logs de diferent mida per verificar empíricament que el comportament és lineal $O(N)$. Les proves s'han fet amb el generador de logs sintètics inclòs al projecte.

Amb un log de 47 línies (el de demostració), el sistema processa totes les línies, detecta 5 IPs atacants i identifica 1 subxarxa compromesa en menys de 10 milisegons. El cost per línia es manté constant independentment de la mida del log, confirmant el comportament lineal teòric.

3. Propostes de millora

- **Optimitzar `contar_ip()`:** ara recorre tota la finestra $O(V)$. Es podria mantenir un comptador per IP per fer-ho en $O(1)$, a canvi d'usar una mica més de memòria.
- **Afegir més analitzadors:** gràcies al polimorfisme, afegir nous tipus de detecció (per exemple, peticions web sospechoses) només requereix crear una nova classe filla sense tocar el codi existent.
- **Lectura en temps real:** ara llegim l'arxiu sencer de cop. Es podria llegir en directe (estil tail -f) per detectar atacs mentre passen.